Yes. We can change the backend to use **FAISS only** and **Gemini 2.5 Flash only**.

So the final stack will be:

```
Python
Flask
PyPDF
LangChain
FAISS               ← Vector Database
SentenceTransformers ← Embeddings
Gemini 2.5 Flash    ← LLM
```

We will **remove ChromaDB completely**.

One important distinction:

```
FAISS ≠ Embedding Model ≠ LLM
```

They have different jobs:

```
PDF
 ↓
Text Extraction
 ↓
Chunking
 ↓
Embedding Model
 ↓
FAISS
 ↓
Retriever
 ↓
Gemini 2.5 Flash
 ↓
Answer
```

For Gemini, use your API key in `.env` rather than hard-coding it.

1. Updated project structure

Use this:

```
real_estate_rag/
│
├── app.py
├── requirements.txt
├── .env
├── .gitignore
│
├── documents/
│   │
│   ├── property_A/
│   │   ├── property_details.pdf
│   │   ├── rental_agreement.pdf
│   │   └── inspection_report.pdf
│   │
│   └── property_B/
│       ├── property_details.pdf
│       ├── rental_agreement.pdf
│       └── inspection_report.pdf
│
├── vector_db/
│   └── faiss_index/
│
├── rag/
│   ├── document_loader.py
│   ├── text_splitter.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── retriever.py
│   └── prompts.py
│
├── features/
│   ├── qa.py
│   ├── summarization.py
│   └── comparison.py
```

```
│
└── utils/
    └── preprocessing.py
```

The important change is:

```
vector_db/
└── faiss_index/
```

instead of:

```
vector_db/
└── chroma_db/
```

2. Install the correct packages

Remove the Chroma-related packages from your project.

Install:

```
pip install flask
pip install flask-cors
pip install python-dotenv
pip install pypdf
pip install langchain
pip install langchain-community
pip install langchain-text-splitters
pip install faiss-cpu
pip install sentence-transformers
pip install google-generativeai
```

Then:

```
pip freeze > requirements.txt
```

You **do not need**:

```
chromadb
langchain-chroma
```

for this project.

3. .env

Create:

`.env`

Use:

```
GEMINI_API_KEY=your_gemini_api_key_here
```

Do not write:

```
GEMINI_API_KEY = "actual-key"
```

inside your Python files.

4. .gitignore

Create:

`.gitignore`

Put:

```
venv/
.env
__pycache__/
vector_db/
*.pyc
```

This prevents your Gemini API key and FAISS files from accidentally going to GitHub.

5. Step 1 — PDF loader

Keep:

rag/document_loader.py

Use:

```
import osfrom pypdf import PdfReaderfrom utils.preprocessing import
clean_textdef load_pdf(file_path):    reader = PdfReader(file_path)
pages = []    for page_number, page in enumerate(        reader.pages,
start=1    ):        text = page.extract_text()        if text:
text = clean_text(text)
pages.append({                "text": text,                "page":
page_number            })    return pagesdef
load_all_documents(documents_folder="documents"):    all_documents =
[]    if not os.path.exists(documents_folder):
```

6. Step 2 — Text preprocessing

utils/preprocessing.py

```
import redef clean_text(text):    text = re.sub(        r"\s+",
" ",        text    )    return text.strip()
```

7. Step 3 — Chunking

rag/text_splitter.py

```
from langchain_text_splitters import
(    RecursiveCharacterTextSplitter)def split_documents(documents):
splitter = RecursiveCharacterTextSplitter(        chunk_size=800,
chunk_overlap=100    )    chunks = []    for document in documents:
split_texts =
splitter.split_text(            document["text"]        )        for
```

```
chunk in split_texts:
chunks.append({                "text": chunk,
"metadata": {                   "property": document["property"],
"source": document["source"],                   "page":
document["page"]               }           })    return chunks
```

8. Step 4 — Embeddings

Here we're using a local embedding model.

`rag/embeddings.py`

```
from langchain_community.embeddings import
(    HuggingFaceEmbeddings)def get_embedding_model():    return
HuggingFaceEmbeddings(        model_name="sentence-transformers/all-
MiniLM-L6-v2"    )
```

The process is:

```
Text
 ↓
all-MiniLM-L6-v2
 ↓
Vector
```

Example:

```
"Security deposit is ₹1,00,000"

        ↓

[0.124, -0.283, 0.512, ...]
```

9. Step 5 — FAISS vector database

This is the major change from the previous version.

Create:

rag/vector_store.py

Use:

```
import osfrom langchain_community.vectorstores import FAISSfrom
langchain_core.documents import Documentfrom rag.embeddings import
get_embedding_modelFAISS_PATH = "vector_db/faiss_index"def
create_vector_store(chunks):    embedding_model =
get_embedding_model()    documents = []    for chunk in chunks:
document = Document(            page_content=chunk["text"],
metadata=chunk["metadata"]        )        documents.append(document)
vector_store = FAISS.from_documents(        documents,
embedding_model    )    os.makedirs(        "vector_db",
exist_ok=True    )
```

Now the architecture is:

```
Chunks
   ↓
Embeddings
   ↓
FAISS
   ↓
vector_db/faiss_index/
```

10. What gets stored in FAISS?

Conceptually:

```
FAISS
│
├── Vector 1
│    └── Property A / rental agreement / page 1
│
├── Vector 2
│    └── Property A / rental agreement / page 2
│
```

```
├── Vector 3
│    └── Property B / rental agreement / page 1
│
└── Vector 4
     └── Property B / inspection report / page 2
```

The metadata is associated with each document.

This is what will allow us to eventually distinguish:

```
Property A
```

from:

```
Property B
```

11. Step 6 — Retriever

```
rag/retriever.py
```

```
from rag.vector_store import load_vector_storedef
retrieve_documents(    question,    k=4):    vector_store =
load_vector_store()    documents =
vector_store.similarity_search(        question,        k=k    )
return documents
```

User:

```
What is the security deposit?
```

Retriever:

```
Question
   ↓
Embedding
   ↓
FAISS similarity search
   ↓
Top 4 relevant chunks
```

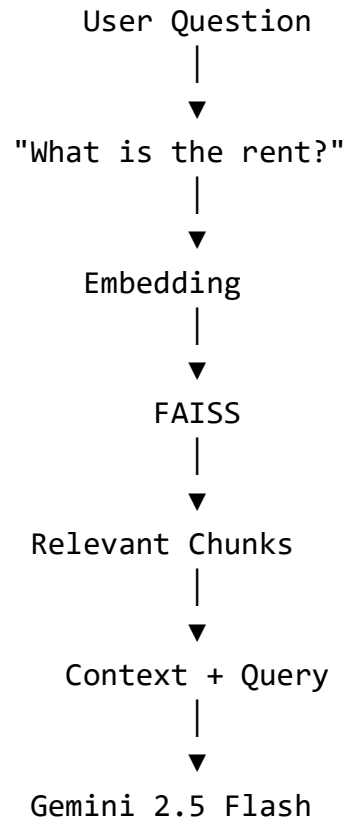
12. Step 7 — Gemini 2.5 Flash

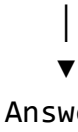
Now we use **Gemini 2.5 Flash as the LLM**.

features/qa.py

```
import osimport google.generativeai as genaifrom dotenv import
load_dotenvfrom rag.retriever import
retrieve_documentsload_dotenv()genai.configure(    api_key=os.getenv("
GEMINI_API_KEY"))model = genai.GenerativeModel(    "gemini-2.5-
flash")def answer_question(question):    documents =
retrieve_documents(        question    )    context =
"\n\n".join(        document.page_content        for document in
documents    )    prompt = f"""You are a Real Estate Document
Intelligence assistant.
```

13. Your Q&A pipeline is now





This is your actual **RAG pipeline**.

14. Step 8 — Prompt file

rag/prompts.py

```
QA_PROMPT = """You are a Real Estate Document Intelligence
assistant.Use ONLY the provided document
context.Context:{context}Question:{question}Rules:1. Do not invent
information.2. Do not use outside knowledge.3. If the answer is
unavailable, say:   "Information not found in the uploaded
documents."4. Give a concise answer."""
```

Later we will create:

SUMMARY_PROMPT

and:

COMPARISON_PROMPT

inside the same file.

15. Step 9 — Flask backend

app.py

```
from flask import Flask, jsonify, requestfrom flask_cors import
CORSfrom dotenv import load_dotenvfrom rag.document_loader import
load_all_documentsfrom rag.text_splitter import split_documentsfrom
rag.vector_store import create_vector_storefrom features.qa import
answer_questionload_dotenv()app =
```

```
Flask(__name__)CORS(app)@app.route(    "/api/health",
methods=["GET"])def health():    return jsonify({        "status":
"success",        "message": "Real Estate RAG Backend is
running"    })@app.route(    "/api/index",    methods=["POST"])def
index_documents():
```

16. Test the backend

Run:

```
python app.py
```

You should see:

```
Running on http://127.0.0.1:5000
```

Test 1 — Health

```
GET /api/health
```

Response:

```
{
    "status": "success",
    "message": "Real Estate RAG Backend is running"
}
```

17. Test 2 — Index documents

Send:

```
POST /api/index
```

You don't need a body.

Expected:

```
{
    "status": "success",
    "message": "Documents indexed successfully",
    "documents": 1,
    "chunks": 3
}
```

Now your folder should contain:

```
vector_db/
└── faiss_index/
    ├── index.faiss
    └── index.pkl
```

🎯 **That proves your documents have been embedded and stored in FAISS.**

18. Test 3 — Ask a question

Send:

```
POST /api/qa
```

JSON:

```
{
    "question": "What is the security deposit?"
}
```

Expected:

```
{
    "status": "success",
    "question": "What is the security deposit?",
    "answer": "The security deposit is ₹1,00,000.",
    "sources": [
        {
```

```
            "property": "property_A",
            "source": "rental_agreement.pdf",
            "page": 1
        }
    ]
}
```

19. Your technology roles are now very clear

This is important for your project explanation/interview.

PDF

`PyPDF`

Extracts the document text.

Chunking

`RecursiveCharacterTextSplitter`

Breaks large documents into smaller pieces.

Embeddings

`all-MiniLM-L6-v2`

Converts text into vectors.

Vector database

`FAISS`

Stores and searches those vectors.

Retriever

`FAISS similarity search`

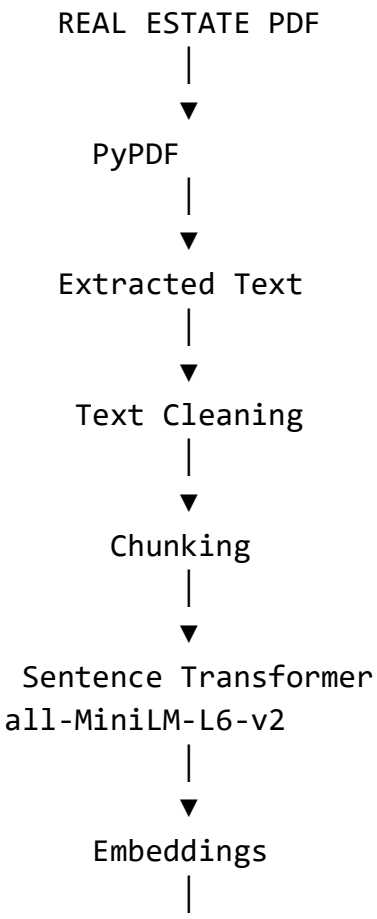
Finds the most relevant chunks.

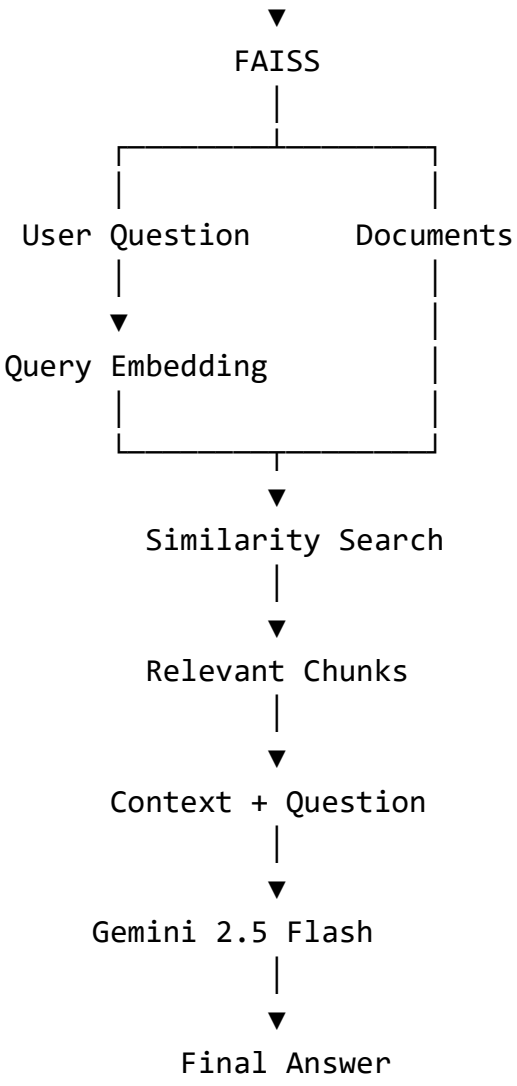
LLM

`Gemini 2.5 Flash`

Uses retrieved context to generate the answer.

20. Complete RAG architecture for YOUR project



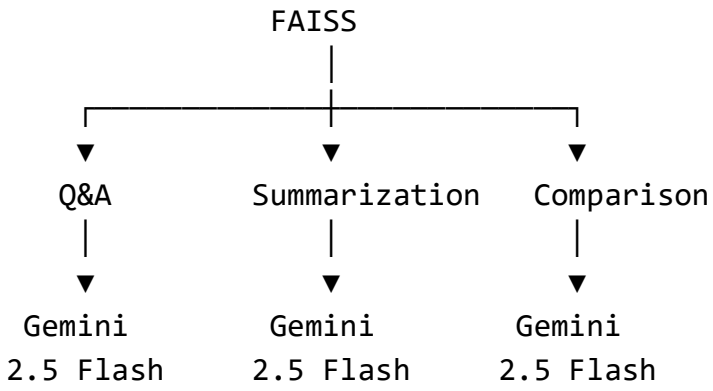


21. And later, the same FAISS database supports all 3 features

You don't create:

```
FAISS_QA
FAISS_SUMMARY
FAISS_COMPARISON
```

Instead:



So **one FAISS knowledge base + Gemini 2.5 Flash** can support the entire application.

For comparison, when we add Property B, the same FAISS index will contain metadata such as:

```
property_A + rental_agreement + page 1
property_A + inspection_report + page 2
property_B + rental_agreement + page 1
property_B + inspection_report + page 2
```

Then we use metadata filtering/retrieval to get Property A and Property B separately before sending both contexts to **Gemini 2.5 Flash**.

One correction from the earlier code

Because you're specifically standardizing on **Gemini 2.5 Flash + FAISS**, use the architecture above consistently and **remove all ChromaDB code/dependencies**. We should build the project incrementally from here rather than mixing the two vector databases.